Databricks Developer Workflow

This deck explains the practical execution model behind everyday Databricks development, from the notebook runtime and language mixing to data access and temporary views. It focuses on the read-transform-query pattern so developers can reason clearly about execution, state, and the boundary between notebook code and distributed Spark jobs.

DEVELOPER WORKFLOW | SPARK + NOTEBOOKS

RECAP FOUNDATIONS

Recap: The Platform You Build On

Before we go deeper into developer workflow, it helps to re-anchor on the core Databricks building blocks. The **control plane** manages orchestration, configuration, and workspace services. The **data plane** is where clusters run and where Spark actually processes data. **Clusters** provide the compute environment for notebooks and jobs. **DBFS** offers a Databricks-managed file abstraction often used to reference files and persisted outputs. Together, these concepts explain where code lives, where it executes, and where data is read from or written to.

Control Plane

Workspace management, orchestration, notebook interface, permissions, and service coordination.

Data Plane

Where compute resources run and Spark workloads execute against data.

Clusters

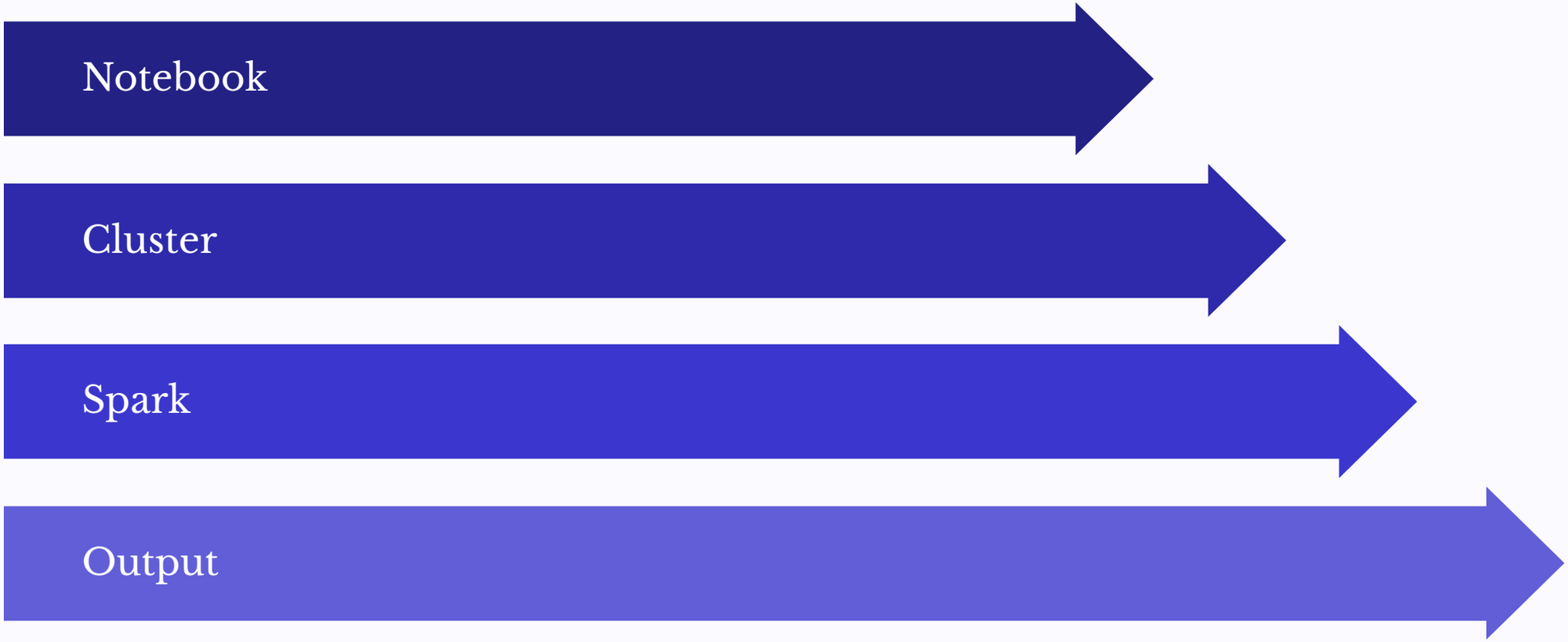
Provisioned compute used by interactive notebooks and scheduled jobs.

DBFS

A familiar filesystem-like access layer used throughout Databricks workflows.

EXECUTION FLOW MENTAL MODEL

How Code Executes in Databricks



The notebook is the authoring surface, not the compute engine. When you run a cell, the command is sent to an attached cluster. The cluster provides the runtime environment, while Spark plans and executes distributed work where relevant. Results then return to the notebook as displayed tables, logs, charts, or persisted data. This model is essential because it separates **writing code** from **executing code**. A notebook may feel local, but much of the real work happens in a remote, distributed environment.

NOTEBOOK MODEL

STATEFULNESS

Notebook Execution Is Cell-Based and Stateful

What this means

Each cell can be run independently, but the notebook session retains variables, imports, cached data, and temporary objects for as long as the attached cluster session remains active.

Cell execution

Cells run one command block at a time, which supports exploration and incremental development.

Shared session state

Objects created in one cell may be reused later without redefinition, provided the session has not been reset.

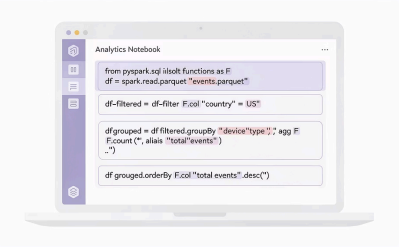
Order matters

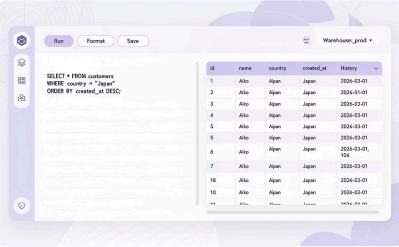
Running cells out of sequence can produce confusing results when notebook state no longer matches the visible code order.

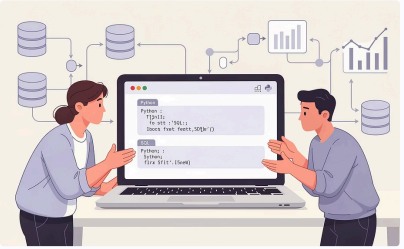
For developers, this is powerful but risky: notebooks support rapid iteration, yet hidden state can mask bugs. A reliable habit is to think not only about what a cell contains, but also what the current session already remembers.

LANGUAGES | MIXED WORKFLOWS

Databricks Supports Multi-Language Development







Python

Common for DataFrame operations, ETL logic, reusable functions, and integration with the wider PySpark ecosystem.

SQL

Ideal for querying views and tables, validating outputs, and expressing transformations in a declarative form.

Mixed workflows

Many teams read and transform with Python, then expose the result to SQL for inspection, joining, or downstream analysis.

The key idea is not language choice in isolation, but **interoperability**. Databricks lets developers move between procedural and declarative styles in one workspace, which is one of the reasons notebook-based Spark development feels so fluid in practice.

NOTEBOOK MECHANICS | MAGIC COMMANDS

Magic Commands Are a Workflow Bridge

Commands such as `%python` and `%sql` are not part of Python or SQL syntax themselves. They are notebook directives that tell Databricks which interpreter should run the cell. This makes them a workflow tool rather than a language feature.

%python

Use Python for reading data, transforming DataFrames, and managing programmatic logic.

Context switch

The notebook changes execution context without forcing you into a separate file or tool.

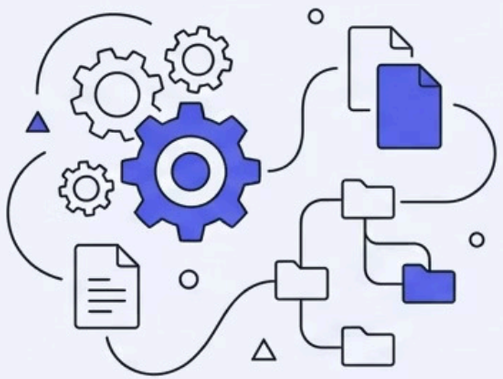
%sql

Query views or tables immediately to validate the result of previous DataFrame work.

> A useful mental model: magic commands do not change Spark itself; they change how you communicate with it inside the notebook.

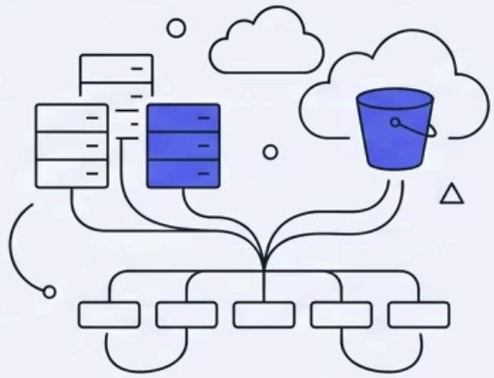
DATA ACCESS | STORAGE PATHS

Common Data Access Patterns



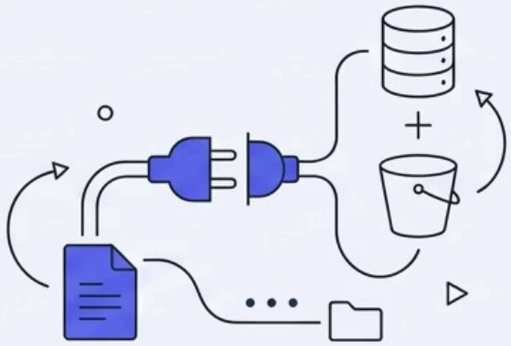
DBFS (Databricks File System)

Platform-friendly file access within Databricks workflows. Local filesystem simulation for code.



S3 (Underlying Cloud Object Storage)

Durable, long-term cloud object storage. Acts as the persistent data layer.



Mounts (Convenience Layer)

Exposes external storage through filesystem paths. A convenience for direct data interaction.

Developers usually interact with storage through a path, but not every path represents the same abstraction. **DBFS** provides a Databricks-oriented access layer. **S3** is the underlying object store in AWS-based environments and is often the durable system of record. **Mounts** provide a convenience pattern that exposes external storage through familiar filesystem-like paths. Understanding the distinction helps when debugging permissions, portability, and environment-specific behaviour.

BRIDGE | TEMPORARY VIEWS

Temporary Views Connect DataFrames and SQL

A temporary view lets you take a DataFrame created in code and expose it to SQL within the current session. This is one of the most useful patterns in Databricks because it creates a clear bridge between transformation logic and interactive querying.

01

Create DataFrame

Read and shape data using Python and the DataFrame API.

02

Register view

Publish the in-memory result as a temporary SQL-accessible name.

03

Query with SQL

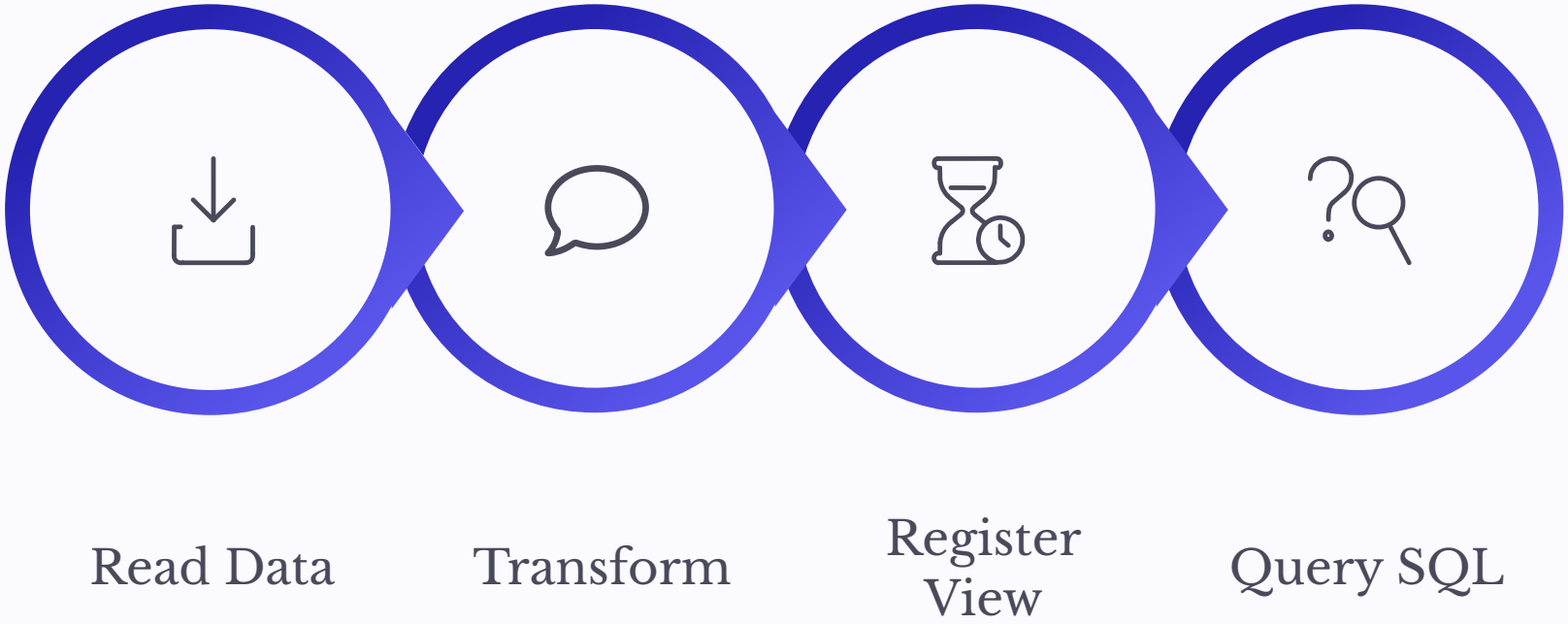
Inspect, filter, aggregate, or join the result using familiar SQL syntax.

The view is session-scoped rather than permanently stored metadata. That makes it excellent for development, validation, and iterative exploration, but not a substitute for durable production tables.

CORE PATTERN

HANDS-ON WORKFLOW

The Standard Developer Workflow



This pattern appears again and again in real notebook work. Developers **read** data from storage, **transform** it with Spark DataFrames, **register** the result as a temporary view, and then **query** it with SQL. The value of this sequence is that it combines the strengths of both APIs: Python for expressive transformation logic and SQL for fast inspection and analytical clarity. Once this pattern feels natural, notebook development becomes much more predictable and efficient.

TRANSITION NEXT TOPIC

We have established the workflow around notebooks, interpreters, storage, and temporary views. The next step is to focus on the **DataFrame** itself: the core abstraction most Databricks developers use to model, transform, and optimise data processing in Spark. This is where API design, execution planning, and developer ergonomics come together.

Why it matters

DataFrames sit at the centre of modern Spark development.

What comes next

Schema, transformations, lazy execution, and practical usage patterns.